

Présentation Mini-Projet BII

Polytech Paris-Saclay, Département EES, 3ème année

Nicolas Lermé

`nicolas.lerme@universite-paris-saclay.fr`

Année 2025–2026



Plan de la présentation

1 Introduction

- Cadre, objectif et rappel du jeu
- Évaluation, rendu et oral
- Leaderboard

2 Code et données

- Structure du projet fourni
- Instances, pools et données

3 Mini-cours d'optimisation

- Notations communes
- Heuristiques déterministes
- Heuristiques stochastiques
- Heuristiques à population et hybrides
- Expérimentation et réglage

4 Conclusion

Plan

- 1 Introduction
 - Cadre, objectif et rappel du jeu
 - Évaluation, rendu et oral
 - Leaderboard
- 2 Code et données
 - Structure du projet fourni
 - Instances, pools et données
- 3 Mini-cours d'optimisation
 - Notations communes
 - Heuristiques déterministes
 - Heuristiques stochastiques
 - Heuristiques à population et hybrides
 - Expérimentation et réglage
- 4 Conclusion

Objectifs et compétences du mini-projet

Fil directeur (2 groupes)

Implémenter des algorithmes pour résoudre un problème combinatoire (Boggle).

Compétences visées

- Mise en pratique des connaissances acquises durant le cours.
- Appropriation d'un code existant (pointeurs, structures, I/O, etc.).
- Acquisition de quelques notions d'optimisation.
- Utilisation d'une plateforme de dépôt.

Déroulement du module

Séquençage

- ❶ 2h de TD : présentation du sujet.
- ❷ 1^{er} TP de 4h : prise en main du code, compilation, tests, premières heuristiques.
- ❸ 2^e TP de 4h : amélioration, comparaison, **évaluation écrite**.

Équilibre attendu

Le projet final et l'évaluation comptent tous deux :

- L'un mesure le travail construit,
- L'autre la compréhension réelle et l'autonomie.

Organisation pratique des séances

TD (aujourd'hui)

- Problème, objectifs, attentes.
- Structuration du code et format des données.
- La plateforme de dépôt.

Premier TP

- Compiler, exécuter, comprendre le code et les données.
- Evaluer la baseline actuelle et investiguer des pistes d'amélioration.

Second TP

- Boucle de feedback implémentation <-> évaluation des performances.
- Evaluation écrite autonome sur 2h.

Évaluation et note finale

Composantes

- 1 **Programme** : qualité du code, algorithmes implémentés, gestion mémoire.
- 2 **Compte-rendu** : clarté, justification des choix, analyse critique des résultats.
- 3 **Évaluation écrite** : à définir.

Qualité des livrables attendus

Programme

L'archive devra contenir le programme complet (fichiers sources `*.h` et `*.cpp`), une documentation, des commentaires clairs et explicatifs, la solution Visual Studio et le `textttMakefile` (évaluation), les données et autres fichiers textes.

Compte-rendu

Un PDF qui présente le problème, l'architecture du code existant, le format des données, les algorithmes testés, les résultats obtenus sur les données fournies et une discussion critique (avantages/inconvénients, pistes d'amélioration, etc.).

Modalités de rendu

Une archive ZIP contenant le programme et le compte-rendu devra être déposée sur la branche réseau de l'école. Les livrables non déposés ne seront pas évalués.

Consignes importantes

Règles de travail

- 1 Travail individuel obligatoire.
- 2 Plagiat interdit (donc sanctionné).
- 3 Commentaires et documentation.
- 4 Code compilable sans erreurs ni warnings.

Règles pendant l'évaluation

- Pas d'accès à Internet (câble RJ45 débranché, clé USB obligatoire).
- Pas de smartphone ni montres connectées.
- Ressources externes et agents IA interdits.
- Sortie autorisée qu'au bout d'1h et accompagnée au-delà.

Contexte : c'est quoi Boggle ?

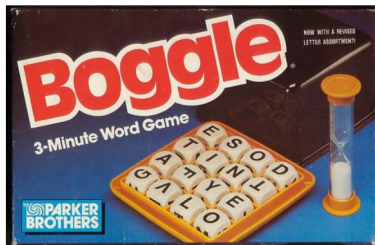


Figure 1 – Boîte de d'origine du jeu Boggle.

Infos

- Jeu multi-joueurs inventé par Allan Turoff et conçu par Bill Cooke.
- Commercialisé par la marque Hasbro depuis 1972.
- Contenu du jeu : boîte de 4 rangées de 4 dés à 6 faces + sablier.

Principe du jeu

- Série de manches alternant phase de jeu / phase de comptage des points.
- Phase de jeu : les dés sont mélangés et chaque joueur doit former un maximum de mots dans un temps imparti selon un dictionnaire de référence.
- Phase de comptage des points : une fois le temps écoulé, chaque joueur révèle les mots formés et applique les règles de comptage des points (voir Tableau 2).
- Le gagnant est celui qui a remporté le plus de manches.

Taille du mot	3	4	5	6	7	8+
Points	1	1	2	3	5	11

Figure 2 – Mécanisme de comptage des points.

Boggle direct : chemins valides

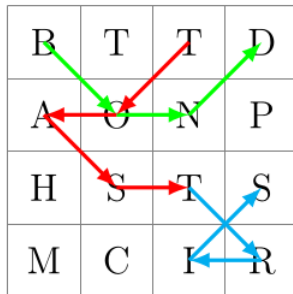


Figure 3 – Exemple d'une grille de jeu et de mots possibles.

Règles du jeu

- Une case peut servir dans plusieurs mots différents.
- Dans un même mot, une case ne doit pas être utilisée deux fois.
- Les déplacements horizontaux, verticaux et diagonaux sont autorisés.
- Un mot n'est autorisé que s'il appartient au dictionnaire de référence.
- Un mot n'est compté que s'il contient au moins 3 lettres.
- Les accents et la casse ne sont pas pris en compte.
- Les noms propres, les noms étrangers, les abréviations, les mots composés ne sont pas autorisés.
- Les mots féminins, pluriels et les conjugaisons sont autorisés.
- Un mot apparaissant plusieurs fois n'est compté qu'une fois.
- Les réductions sont autorisées (ex : danse).
- Le Boggle direct consiste à trouver tous les mots valides d'une grille donnée.

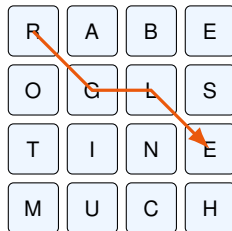
Le mini-projet en une phrase

Entrée

Une instance fournit une grille de départ et les données nécessaires pour évaluer les mots trouvés : dictionnaire, fréquences, règles de score et contraintes d'exécution.

Sortie attendue

Votre programme produit une grille améliorée implicitement par ses résultats et affiche trois indicateurs : score final, nombre final de mots, courbure moyenne.



Formulation intuitive du problème

État courant

Une grille est notée G . Elle contient des lettres dans des cases. À chaque étape, on peut produire une grille voisine G' en modifiant une ou plusieurs lettres.

Objectif

Trouver une suite de grilles

$$G_0 \longrightarrow G_1 \longrightarrow \cdots \longrightarrow G_T$$

qui améliore autant que possible la valeur

$$\text{Score}(G_T).$$

Point important

Le changement de lettres est autorisé. On ne cherche pas seulement à lire au mieux une grille imposée : on cherche à construire une grille plus favorable.

Pourquoi autoriser le changement des lettres ?

Version plus contrainte

La grille est fixée. L'algorithme doit seulement trouver les mots présents.

- intérêt algorithmique limité pour l'optimisation ;
- score rapidement plafonné ;
- peu de marge pour comparer des heuristiques.

Version R-Boggle

La grille peut être modifiée. L'algorithme choisit où investir son budget de calcul.

- espace de recherche beaucoup plus riche ;
- compromis exploration / exploitation ;
- meilleures opportunités de gains substantiels.

Optimiser revient à décider **quoi modifier**, **quand accepter** et **quand s'arrêter**.

Rappel : grille, voisinage et chemins

Grille

On note $G \in \mathcal{A}^{m \times n}$, où $\mathcal{A}$ est l'alphabet autorisé. La case (i, j) contient la lettre G_{ij} .

Voisinage de cases

Deux cases sont voisines si elles se touchent horizontalement, verticalement ou diagonalement. Le voisinage d'une case c est noté $\mathcal{N}(c)$.

Chemin valide

Un chemin $p = (c_1, \dots, c_k)$ est valide si $c_{\ell+1} \in \mathcal{N}(c_\ell)$ et si aucune case n'est utilisée deux fois dans le même mot.

Dictionnaire, mots trouvés et score

Mot lu sur un chemin

Pour un chemin $p = (c_1, \dots, c_k)$, le mot lu est

$$w_G(p) = G_{c_1} G_{c_2} \cdots G_{c_k}.$$

Ensemble des mots trouvés

Si $\mathcal{D}$ est le dictionnaire, on note

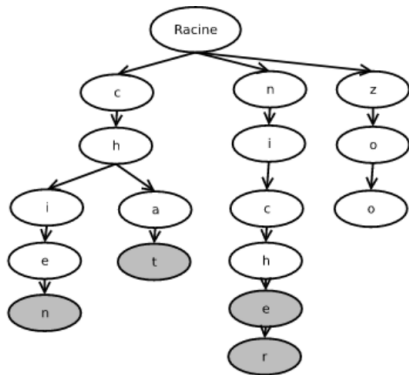
$$\mathcal{W}(G) = \{w_G(p) : p \in \mathcal{P}(G), w_G(p) \in \mathcal{D}\}.$$

Score

Avec une fonction de gain $q(w)$,

$$\text{Score}(G) = \sum_{w \in \mathcal{W}(G)} q(w).$$

Boggle direct : dictionnaire sous forme d'arbre



Intuition

Un arbre lexicographique permet d'arrêter immédiatement une exploration dès que le préfixe courant ne peut mener à aucun mot.

Gain attendu

On évite de développer des chemins inutiles dans la grille : c'est essentiel pour évaluer beaucoup de grilles pendant l'optimisation.

Le coeur combinatoire

Explosion du nombre de grilles

Si la grille possède mn cases et $\#\mathcal{A}$ lettres possibles, le nombre de grilles possibles est

$$(\#\mathcal{A})^{mn}.$$

Même pour une petite grille, l'énumération exhaustive est hors de portée.

Conséquence

Le projet n'est pas de prouver que l'on a trouvé l'optimum global. Le projet est de concevoir une procédure efficace qui produit de bonnes grilles dans un budget de temps limité.

Ce qui est évalué

Performance calculée

- score agrégé ;
- nombre moyen de mots ;
- courbure moyenne ;
- nombre d'instances traitées correctement ;
- temps d'exécution.

Qualité de la démarche

- choix d'heuristiques justifié ;
- protocole expérimental propre ;
- comparaison honnête des variantes ;
- analyse des limites et des échecs.

Contrat d'exécution du programme

Commande utilisée par le juge

Le programme C++ est lancé avec le nom du fichier d'instance en argument :

```
./main data/<nom_instance>.txt
```

Sortie standard attendue

À la fin de chaque exécution, trois lignes doivent être présentes :

Indicateur	Format attendu
score	final score -> ...
nb mots	final nb words -> ...
courbure	final mean curvature -> ...

Rendu et oral : ce qui doit être raconté

Rendu

Votre dépôt doit contenir le code compilable, les scripts de génération d'archive fournis et les fichiers nécessaires au projet, sans données privées du serveur.

Oral

Présenter une histoire algorithmique :

- ① point de départ ;
- ② heuristiques testées ;
- ③ paramètres importants ;
- ④ résultats observés ;
- ⑤ ce qui a échoué et pourquoi ;
- ⑥ amélioration la plus rentable.

MAAT : à quoi sert le leaderboard ?

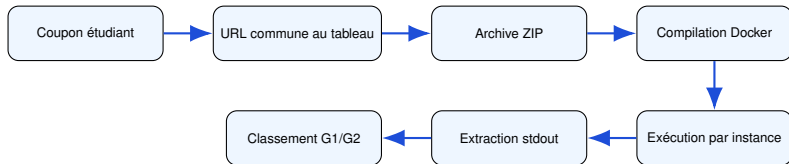
MAAT en pratique

MAAT est une plateforme locale de dépôt et d'évaluation automatique. Elle compile les dépôts C/C++ dans Docker, exécute chaque programme sur les instances configurées, extrait les indicateurs de la sortie standard et met à jour les classements.

Intérêt pédagogique

- feedback rapide sur les erreurs de format, compilation et exécution ;
- comparaison dynamique des stratégies ;
- suivi de progression personnelle ;
- support pour l'enseignant : admin, incidents, export CSV, rapports PDF.

Flot complet d'un dépôt



Confidentialité

Le token individuel est un secret. Il ne doit pas être partagé. Le QR code des coupons renvoie aux supports de cours, pas à une URL de soumission personnalisée.

Bundle MAAT : composants utiles

Scripts enseignant

- `manage-maat.sh`
install/start/stop/check
- `manage-tunnel.sh`
start/stop/logs
- génération des PDF
tokens/coupons ;
- réglage automatique de la
parallélisation.

Interface web

- dépôt étudiant ;
- classements par groupe ;
- vue admin et vue synthétique ;
- pause/reprise des dépôts ;
- export CSV et rapports PDF.

Démo UI : dépôt étudiant – consignes

Mini-Projet | R-Boggle

Polytech Paris-Saclay · Bases de l'Informatique Industrielle · 3ème année

[Dépôt](#)[Classement G1](#)[Classement G2](#)[Clair](#)[Sombre](#)[A-](#)[A+](#)[FR](#)[EN](#)

Temps restant TP: 3h 58m 55s

Format attendu pour l'archive ZIP

L'archive doit contenir vos sources C++ et permettre de trouver un fichier main.cpp.

Accepté : src/main.cpp, include/*.h, Makefile, Makefile.common, Makefile.depend, obj/*.o

Compilation : si un Makefile existe, il est utilisé avec reconstruction forcée ; sinon le serveur compile src/*.cpp.

Données : ne mettez pas dict.txt, letters_freq.txt ou les instances dans le ZIP.

```
mon_projet.zip
├── src/
│   ├── main.cpp
│   └── autres_fichiers.cpp
├── include/
│   └── autres_fichiers.h
├── Makefile
├── Makefile.common
├── Makefile.depend
└── obj/*.o
```

Démo UI : règles de dépôt

Format attendu pour la sortie standard

Votre programme doit écrire ces trois lignes, avec des valeurs numériques, dans stdout pour chaque instance.

```
final score -> ...  
final nb words -> ...  
final mean curvature -> ...
```

Générer l'archive avant dépôt

Windows

Allez dans le répertoire submission, puis cliquez sur `make_submission_windows.bat`.

Linux / macOS

Allez dans le répertoire submission, puis cliquez sur `make_submission_linux.sh`.

Règles de dépôt

- Un dépôt accepté toutes les 5 minutes par étudiant.
- Un seul dépôt en attente ou en cours par token.
- Le classement conserve le meilleur dépôt terminé et non annulé.
- Le token est lié au premier poste/navigateur qui l'utilise.
- Dépôts évalués en parallèle : 2.
- Taille maximale du ZIP : 20 Mo.
- Nombre maximal de fichiers : 100.
- Taille maximale décompressée : 80 Mo.
- Temps maximal de compilation : 30 s.
- Temps maximal d'exécution par instance : 180 s.
- Taille maximale conservée par stdout/stderr : 200000 octets.
- Compilation parallèle : `make -j2`.
- Exécution parallèle : jusqu'à 2 thread(s), quota Docker 2 CPU.
- Extensions autorisées : `.c`, `.cc`, `.cpp`, `.cxx`, `.h`, `.hh`, `.hpp`, `.hxx`, `.depend`, `.common`, `.o`.

Démo UI : dépôt étudiant – formulaire et historique

Déposer un travail

Déposez votre archive ZIP. Le serveur compile et évalue votre programme C++ dans un conteneur isolé.

Token individuel

.....

Le token est mémorisé localement dans votre navigateur.

Nom de l'heuristique

baseline

Cette valeur est mémorisée localement dans votre navigateur.

Archive ZIP

Choisir un fichier

Aucun ...er choisi

Déposer

Historique des dépôts

auto-refresh

Dépôts connus par ce navigateur. Cliquez sur une ligne pour ouvrir le détail.

Légende des statuts Q En file RUN En cours OK Terminé ANNULE Annulée CE Erreur de compilation TLE Temps limite dépassé ZIP Archive ZIP invalide

MISS Fichier attendu manquant

Dépôt	Statut	Heuristique	Score	#Instances ok	#Instances échec	Date	Rapport PDF
-------	--------	-------------	-------	---------------	------------------	------	-------------

Aucun dépôt connu dans ce navigateur.

Démo UI : classement global

Mini-Projet | R-Boggle

Polytech Paris-Saclay · Bases de l'Informatique Industrielle · 3ème année

Dépôt
Classement G1
Classement G2

Clair
Sombre
A-
A+
FR
EN

⌚ Temps restant TP: 3h 25m 23s

Classement global

Groupe G1 · Meilleur dépôt terminé et non annulé par étudiant. Mise à jour automatique.

Vos statistiques · Dépôts: 1 · Évolution: 4 566 298 · Meilleur score personnel: 4 566 298 · Rang personnel: 1 · Écart au premier: +0 · Écart avec la précédente: —

Légende des statuts Q En file RUN En cours OK Terminé ANNULÉ Annulée

Rang	Qui ?	Heuristique	Score	Nombre moyen de mots	Courbure moyenne	#Instances ok	#Instances échec	Temps total	Dépôt	Date
1	 	baseline	4 566 298	4 185,5	0,423767	18/18	0/18	205,563 s	000002	03-05-2026

Démo UI : classement par instance

Classement par instance								
Légende des statuts 0 En file RUN En cours OK Terminé ANNULÉ Annulée								
Instance	Qui ?	Heuristique	Score	Nombre de mots	Courbure moyenne	Temps total	Dépôt	Date
public_1_easy_4_100_100.txt		baseline	5 093	184	0,505616	1,984 s	000002	03-05-2026 @ 10:49:15
public_2_easy_4_101_100.txt		baseline	12 359	347	0,454055	1,824 s	000002	03-05-2026 @ 10:49:15
public_3_easy_4_102_1000.txt		baseline	22 323	537	0,445627	2,197 s	000002	03-05-2026 @ 10:49:15
public_4_easy_4_103_1000.txt		baseline	21 342	533	0,454744	2,235 s	000002	03-05-2026 @ 10:49:15
public_5_easy_4_104_5000.txt		baseline	22 392	562	0,439475	3,067 s	000002	03-05-2026 @ 10:49:15
public_6_easy_4_105_5000.txt		baseline	35 779	765	0,463177	3,606 s	000002	03-05-2026 @ 10:49:15
public_7_medium_8_200_100.txt		baseline	91 189	2 042	0,416503	2,483 s	000002	03-05-2026 @ 10:49:15
public_8_medium_8_201_100.txt		baseline	73 854	1 805	0,398115	2,156 s	000002	03-05-2026 @ 10:49:15
public_9_medium_8_202_1000.txt		baseline	271 301	4 544	0,406273	4,71 s	000002	03-05-2026 @ 10:49:15
public_10_medium_8_203_1000.txt		baseline	329 892	5 199	0,406191	5,093 s	000002	03-05-2026 @ 10:49:15

Démon UI : détail d'un dépôt

Mini-Projet | R-Boggle

Polytech Paris-Saclay · Bases de l'Informatique Industrielle · 3ème année

DépôtClassement G1Classement G2

ClairSombreA-A+FREN

 **Temps restant TP: 3h 24m 02s**

Dépôt #000002

OK **auto-refresh**

Évaluation terminée.

Rapport PDF **Annuler le dépôt**

Étudiant : Rayane ISKOUNENE 🟡 · Groupe G1

Dépôt : heuristique=baseline · Date : 03-05-2026 @ 10:49:15

4 566 298
score agrégé

18/18
#Instances ok

0/18
#Instances échec

205,563 s
Temps total

Démo UI : détail par instance

Détail par instance

Légende des statuts En file En cours Terminé / Instance valide Annulée Erreur de compilation Erreur d'exécution

Temps limite dépassé Archive ZIP invalide Fichier attendu manquant Sortie invalide Erreur interne

Instance	Statut	Score	Nombre de mots	Courbure moyenne	Temps	Message	Sorties
public_1_easy_4_100_100.txt		5 093	184	0,505616	1,984 s		
public_2_easy_4_101_100.txt		12 359	347	0,454055	1,824 s		
public_3_easy_4_102_1000.txt		22 323	537	0,445627	2,197 s		
public_4_easy_4_103_1000.txt		21 342	533	0,454744	2,235 s		
public_5_easy_4_104_5000.txt		22 392	562	0,439475	3,067 s		
public_6_easy_4_105_5000.txt		35 779	765	0,463177	3,606 s		
public_7_medium_8_200_100.txt		91 189	2 042	0,416503	2,483 s		
public_8_medium_8_201_100.txt		73 854	1 805	0,398115	2,156 s		
public_9_medium_8_202_1000.txt		271 301	4 544	0,406273	4,71 s		
public_10_medium_8_203_1000.txt		329 892	5 199	0,406191	5,093 s		

Coupons étudiants

Mini-Projet | R-Boggle - Polytech Paris-Saclay

Bases de l'Informatique Industrielle - 3ème année
2025-2026

Hajar AFTISS

Groupe : **G1**

Token : **MRSGHTT6LKCN66JL**

Symbole : 🐼

Ne pas partager ce coupon

URL du serveur affichée au tableau



Supports

Mini-Projet | R-Boggle - Polytech Paris-Saclay

Bases de l'Informatique Industrielle - 3ème année

Informations présentes

- identité de l'étudiant ;
- groupe ;
- token individuel de 16 caractères ;
- symbole ;
- rappel : ne pas partager le coupon ;
- rappel : URL serveur affichée au tableau ;
- QR vers les supports de cours.

Principe

Le coupon aide à s'authentifier, mais il ne doit pas circuler entre étudiants.

Plan

- 1 Introduction
 - Cadre, objectif et rappel du jeu
 - Évaluation, rendu et oral
 - Leaderboard
- 2 **Code et données**
 - Structure du projet fourni
 - Instances, pools et données
- 3 Mini-cours d'optimisation
 - Notations communes
 - Heuristiques déterministes
 - Heuristiques stochastiques
 - Heuristiques à population et hybrides
 - Expérimentation et réglage
- 4 Conclusion

Organisation attendue de l'archive ZIP

Exemple minimal

Élément	Rôle
Makefile	compilation du programme
src/main.cpp	point d'entrée C++
src/*.cpp	implémentations auxiliaires
include/*.h (pp)	déclarations
submission/	scripts pour fabriquer l'archive

À retenir

Le serveur ne juge pas une intention : il compile l'archive et exécute le binaire attendu.
Un bon algorithme mal empaqueté vaut zéro sur le leaderboard.

Ce que le programme reçoit à l'exécution

Argument obligatoire

Votre `main` reçoit le chemin de l'instance :

```
argv[1] = "data/<nom_instance>.txt".
```

Fichiers disponibles

Le serveur ajoute dans le répertoire d'exécution :

- `data/dict.txt` : dictionnaire ;
- `data/letters_freq.txt` : fréquences ou informations de tirage ;
- `data/<nom_instance>.txt` : instance courante.

Squelette robuste pour lire l'argument

Point d'entrée conseillé

```
int main(int argc, char** argv) {  
    if (argc < 2) {  
        std::cerr << "usage: ./main data/<instance>.txt\n";  
        return 1;  
    }  
    std::string instance_file = argv[1];  
    // 1. charger l'instance  
    // 2. lancer l'heuristique  
    // 3. afficher les trois lignes finales  
}
```

Instances publiques et privées

Instances publiques

Elles servent à vérifier votre pipeline, comparer rapidement des variantes et détecter les erreurs grossières de format ou d'exécution.

Instances privées

Elles limitent le sur-apprentissage sur les données visibles. Une stratégie robuste doit rester performante sur des grilles non vues pendant le développement.

Bonne pratique

Optimisez vos idées, pas uniquement un fichier. Une amélioration fiable doit se voir sur plusieurs instances, pas seulement sur un cas favorable.

Pipeline d'évaluation local

- 1 compiler avec les mêmes options que le serveur ;
- 2 exécuter sur quelques instances publiques ;
- 3 vérifier les trois lignes de sortie ;
- 4 mesurer le temps par instance ;
- 5 conserver les meilleurs paramètres dans un fichier ou une constante claire ;
- 6 déposer uniquement une archive propre.

Question utile avant chaque dépôt

Le score a-t-il augmenté parce que l'heuristique est meilleure, ou seulement parce qu'un paramètre a été réglé sur une instance particulière ?

Règles de dépôt à garder en tête

Limites serveur

- au plus 100 fichiers ;
- compilation : 30 secondes ;
- exécution : 180 secondes par instance ;
- taille ZIP et taille décompressée bornées.

Incidents fréquents

- ZIP invalide ;
- fichier attendu manquant ;
- erreur de compilation ;
- timeout ;
- sortie standard incomplète.

Pré-calculs utiles

Dictionnaire

Utiliser une structure de préfixes, par exemple un trie, permet d'arrêter rapidement les chemins qui ne peuvent plus former aucun mot du dictionnaire.

Grille

Pré-calculer le voisinage des cases évite de reconstruire les mêmes listes à chaque évaluation.

Score

Si une modification est locale, chercher à réévaluer seulement ce qui change peut être beaucoup plus rentable que recalculer toute la grille.

Plan

- 1 Introduction
 - Cadre, objectif et rappel du jeu
 - Évaluation, rendu et oral
 - Leaderboard
- 2 Code et données
 - Structure du projet fourni
 - Instances, pools et données
- 3 Mini-cours d'optimisation
 - Notations communes
 - Heuristiques déterministes
 - Heuristiques stochastiques
 - Heuristiques à population et hybrides
 - Expérimentation et réglage
- 4 Conclusion

Notations : état, score, voisinage

État

On note x une solution candidate. Ici, x représente une grille G et éventuellement des choix auxiliaires de l'algorithme.

Fonction objectif

On maximise une fonction $f(x)$, que l'on peut identifier au score de la grille :

$$f(x) = \text{Score}(G).$$

Voisinage

Le voisinage $\mathcal{M}(x)$ est l'ensemble des solutions accessibles en un mouvement : changer une lettre, échanger deux lettres, modifier un petit bloc, etc.

Mouvement et variation de score

Mouvement

Un mouvement m transforme x en $x' = m(x)$.

Gain associé

On note

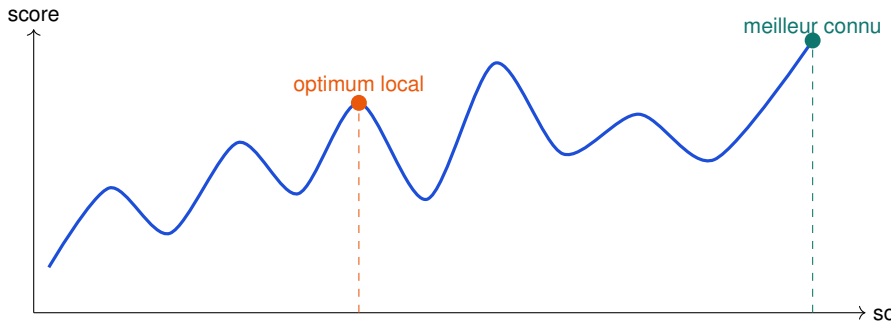
$$\Delta f(x, m) = f(m(x)) - f(x).$$

- $\Delta f > 0$: amélioration ;
- $\Delta f = 0$: plateau ;
- $\Delta f < 0$: dégradation temporaire.

Question centrale

Faut-il accepter uniquement les améliorations ou parfois accepter de dégrader pour sortir d'un optimum local ?

Paysage d'optimisation



Intuition

Une heuristique doit monter vite, mais aussi éviter de rester bloquée trop tôt sur une bonne solution seulement locale.

Budget de calcul

Chaque évaluation coûte quelque chose

Évaluer $f(x)$ peut impliquer une exploration de chemins dans la grille, des recherches dans le dictionnaire et des structures de données non triviales.

Trois budgets typiques

- nombre d'itérations ;
- temps par instance ;
- nombre d'évaluations de score.

Principe pratique

Une heuristique simple mais capable de faire dix fois plus d'essais peut battre une heuristique théoriquement plus sophistiquée mais trop lente.

Baseline : recherche aléatoire

Algorithme

- 1 générer une grille candidate ;
- 2 l'évaluer ;
- 3 conserver la meilleure grille rencontrée ;
- 4 répéter jusqu'à épuisement du budget.

Intérêt

Ce n'est pas forcément une bonne stratégie finale, mais c'est une référence indispensable. Toute heuristique plus complexe doit battre cette baseline clairement.

Hill-climbing : amélioration locale

Règle

À partir de x , choisir un voisin $x' \in \mathcal{M}(x)$ tel que $f(x') > f(x)$, puis remplacer x par x' .

$$x_{t+1} \in \{x' \in \mathcal{M}(x_t) : f(x') > f(x_t)\}.$$

Avantages

Rapide, simple, facile à déboguer, bon premier moteur d'amélioration.

Limite

Bloque dès qu'aucun voisin immédiat n'améliore le score.

First improvement vs best improvement

First improvement

On parcourt les voisins et on accepte le premier qui améliore.

- peu coûteux par itération ;
- trajectoire plus dépendante de l'ordre des tests.

Best improvement

On évalue beaucoup de voisins et on prend le meilleur.

- choix local plus fort ;
- coût plus élevé ;
- risque de convergence prématurée.

Conseil

Comparer les deux avec le même temps total, pas seulement avec le même nombre d'itérations.

Multi-start : recommencer intelligemment

Idée

Lancer plusieurs recherches locales depuis des grilles initiales différentes, puis garder la meilleure solution rencontrée.

$$\text{Best} = \max_{r=1,\dots,R} f(\text{LocalSearch}(x_0^{(r)})).$$

Pourquoi ça marche ?

Une recherche locale est sensible au point de départ. Plusieurs départs augmentent la probabilité de tomber dans le bassin d'attraction d'une bonne solution.

Variable Neighborhood Search

Principe

Changer de voisinage au cours de la recherche :

$$\mathcal{M}_1(x) \subset \mathcal{M}_2(x) \subset \dots \subset \mathcal{M}_K(x).$$

- petit voisinage : exploitation rapide ;
- grand voisinage : perturbation plus forte ;
- retour au petit voisinage quand une amélioration est trouvée.

Exemple R-Boggle

$\mathcal{M}_1$: changer une lettre. $\mathcal{M}_2$: changer deux lettres. $\mathcal{M}_3$: modifier une ligne, une colonne ou un bloc.

Marche aléatoire contrôlée

Idée

Introduire volontairement de l'aléa pour explorer des zones que la recherche gloutonne n'atteint pas.

- choisir une case au hasard ;
- choisir une nouvelle lettre selon les fréquences ;
- évaluer le gain ;
- accepter selon une règle simple.

Usage typique

Bon outil de diversification entre deux phases d'amélioration locale.

Recuit simulé : intuition physique

Image physique

Un métal chauffé peut explorer de nombreuses configurations atomiques. En refroidissant lentement, il se stabilise dans une structure de basse énergie.

Traduction optimisation

On définit une énergie à minimiser :

$$E(x) = -f(x).$$

Maximiser le score revient donc à diminuer l'énergie.

Rôle de la température

Température élevée : on accepte parfois de mauvaises décisions. Température basse : on devient presque glouton.

Recuit simulé : règle d'acceptation

Si le mouvement améliore

Si $\Delta f \geq 0$, on accepte toujours.

Si le mouvement dégrade

Si $\Delta f < 0$, on accepte avec probabilité

$$\mathbb{P}(\text{accepter}) = \exp\left(\frac{\Delta f}{T}\right).$$

- plus la perte $|\Delta f|$ est grande, moins on accepte ;
- plus la température T est grande, plus on accepte ;
- quand $T \rightarrow 0$, la méthode devient presque gloutonne.

Recuit simulé : paramètres essentiels

Paramètre	Rôle	Effet si mal réglé
T_0	température initiale	trop bas : blocage rapide
α	facteur de refroidissement	trop bas : refroidissement brutal
L	essais par palier	trop petit : exploration pauvre
voisinage	type de mutation	trop faible : stagnation
seed	reproductibilité	résultats non comparables

Méthode empirique

Régler T_0 pour accepter au début une proportion raisonnable de mouvements dégradants, puis refroidir progressivement.

Recherche tabou

Idée

Une recherche locale peut cycliser : changer A en B, puis B en A. La mémoire tabou interdit temporairement certains retours en arrière.

Mémoire

On conserve une liste des derniers mouvements ou attributs interdits pendant τ itérations.

Aspiration

Un mouvement tabou peut être autorisé s'il produit la meilleure solution jamais vue.

Recherche tabou : interprétation

- La mémoire courte évite les oscillations immédiates.
- L'acceptation de mouvements non améliorants permet de traverser des vallées.
- Le critère d'aspiration évite que la mémoire bloque une vraie découverte.

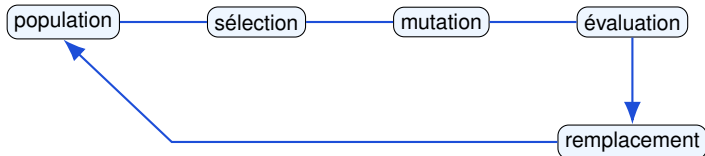
Paramètre principal

La taille tabou τ . Trop petite : cycles. Trop grande : mouvements utiles interdits trop longtemps.

Recherche évolutionnaire

Population

Au lieu de suivre une seule grille, on maintient un ensemble de grilles candidates.



Intérêt

La population maintient plusieurs hypothèses et limite le risque de dépendre d'un seul départ.

Croisement de grilles : attention

Idée séduisante

Combiner deux bonnes grilles en prenant certaines lignes, colonnes ou blocs de chaque parent.

Difficulté

Dans Boggle, la qualité d'une lettre dépend fortement de ses voisines. Un bloc localement bon peut perdre son intérêt lorsqu'il est déplacé ou combiné brutalement.

Conseil

Tester le croisement comme composant optionnel, mais garder une phase de réparation ou d'amélioration locale après croisement.

Elite pool et beam search

Elite pool

Conserver les K meilleures grilles distinctes rencontrées, puis relancer des mutations autour d'elles.

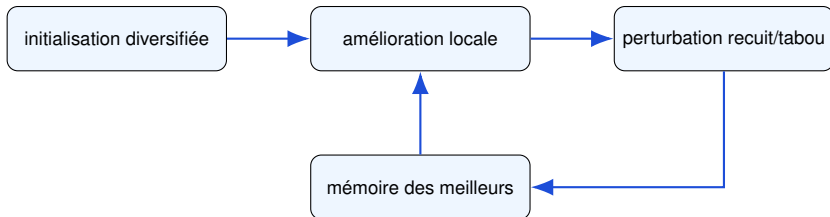
Beam search

À chaque profondeur, garder seulement les K candidats les plus prometteurs au lieu d'explorer tout l'arbre.

Compromis

Un K trop petit donne une recherche trop gloutonne. Un K trop grand disperse le budget.

Architecture hybride recommandée



Lecture

La bonne solution est souvent un assemblage : générer, améliorer, secouer, mémoriser, recommencer.

Comparer correctement deux heuristiques

Comparer à budget identique

Toujours préciser le budget : même temps, même nombre d'instances, même machine, même seed ou même ensemble de seeds.

Mesurer plusieurs dimensions

- score final ;
- stabilité entre répétitions ;
- temps pour atteindre un bon score ;
- sensibilité aux paramètres ;
- nombre d'incidents ou de timeouts.

Ablation : comprendre ce qui aide vraiment

Principe

Une ablation consiste à retirer un composant pour mesurer son effet.

Variante	Composant retiré	Question
A	recuit	sert-il à sortir des optima locaux ?
B	mémoire tabou	évite-t-elle les cycles ?
C	multi-start	améliore-t-il la robustesse ?
D	évaluation incrémentale	change-t-elle le budget utile ?

Pièges classiques

- Optimiser uniquement sur une instance publique.
- Confondre meilleure idée et plus grand temps de calcul.
- Oublier de fixer les seeds dans les expériences.
- Afficher un score correctement mais oublier une des trois lignes finales.
- Faire une heuristique lente qui timeoute sur les instances privées.
- Changer trop de paramètres à la fois, donc ne plus savoir ce qui a aidé.

Journal d'expérience recommandé

Date	Variante	Paramètres	Résultat
J1	greedy	voisinage 1 lettre	rapide, bloqué tôt
J2	multi-start	100 départs	gain stable
J3	recuit	$T_0 = 50, \alpha = 0.98$	meilleur score, plus lent
J4	tabou	$\tau = 30$	moins de cycles

But

Arriver à l'oral avec des décisions justifiées, pas seulement une dernière version du code.

Recette de départ

- 1 Implémenter une évaluation correcte et rapide de la grille.
- 2 Créer une baseline aléatoire reproductible.
- 3 Ajouter un hill-climbing simple.
- 4 Ajouter multi-start.
- 5 Ajouter une perturbation : recuit ou tabou.
- 6 Mesurer sur plusieurs instances.
- 7 Déposer souvent, mais uniquement des versions propres.

Checklist avant dépôt

Technique

- `./main data/<instance>.txt` fonctionne localement ;
- la compilation finit en moins de 30 secondes ;
- l'exécution finit en moins de 180 secondes par instance ;
- les trois lignes finales sont présentes ;
- l'archive est générée par le script fourni.

Scientifique

- la variante a été comparée à une baseline ;
- les paramètres sont notés ;
- les résultats sont reproductibles.

Plan

- 1 Introduction
 - Cadre, objectif et rappel du jeu
 - Évaluation, rendu et oral
 - Leaderboard
- 2 Code et données
 - Structure du projet fourni
 - Instances, pools et données
- 3 Mini-cours d'optimisation
 - Notations communes
 - Heuristiques déterministes
 - Heuristiques stochastiques
 - Heuristiques à population et hybrides
 - Expérimentation et réglage
- 4 Conclusion

Message final

Le projet n'est pas seulement de coder

C'est un exercice complet : modéliser un problème, choisir des mouvements, gérer un budget, comparer des heuristiques et présenter une démarche expérimentale.

Le bon réflexe

Commencer simple, mesurer proprement, complexifier seulement ce qui améliore réellement le résultat.

Références et sources de travail I

Documents de référence du module

- 1 Sujet historique Boggle / Booggle fourni pour le module Bases de l'Informatique Industrielle.
- 2 Dernière version du code source fournie avec les modules `grid`, `node`, `dict`, `words_results`, `direct_solver`, `inverse_solver`, `instance`.

Références algorithmiques classiques

- 1 Recherche locale, hill climbing et redémarrages aléatoires.
- 2 Recuit simulé.
- 3 Tabu search.
- 4 Algorithmes génétiques.